

AI-Enhanced CRM & Ticket Management System

CRM and Ticket Management MVP with AI Assistance and SMTP Notifications

Course	Artificial Intelligence (AI-2002)
Class / Section	BCS-6J
Group	Group 4
Submission Date	2 June 2026

Submitted By

Name	Student ID
Muhammad Sarim Khan Ghouri	23k-0720
Syeda Hamna Uraizee	22k-5112
Sohaib Ahmer	22k-4784

2. Project Overview

The **AI-Enhanced CRM & Ticket Management System** is a minimum viable product developed for TechServe Solutions. It centralizes customer records and support-ticket workflows in a single web application. The system replaces fragmented manual tracking with a structured workspace where support agents can manage their assigned work and managers can monitor the complete service operation.

The core business problem is straightforward: support requests are difficult to manage when customer details, issue descriptions, ownership, progress updates, and resolution records are kept in separate places. Manual processes increase the risk of missed tickets, duplicated effort, slow responses, and poor visibility for decision-makers. TechServe Solutions needs one system that makes the current state of support work visible and traceable.

The implemented MVP addresses this problem with two operational roles:

- **Manager:** can view all customers and tickets, assign agents, delete records, inspect reports, review notification history, and send test notifications.
- **Agent:** can create customers and tickets, view assigned records, add ticket comments, update assigned tickets, use AI assistance, and review scoped dashboard information.

The application includes customer management, ticket management, internal notes, timestamps, activity history, dashboard analytics, manager reports, artificial-intelligence assistance, and SMTP email notifications. AI features help classify tickets, identify customer sentiment, draft reply suggestions, and summarize support records. SMTP messaging makes important ticket events available outside the application through email.

The result is a compact but complete support-management system that demonstrates practical AI integration inside an ordinary business workflow. AI is not presented as a separate experiment. It is embedded where an agent or manager needs it: during ticket intake, issue review, response preparation, escalation, and resolution.

Figure 1. Login page for the TechServe support workspace appears on the following page.

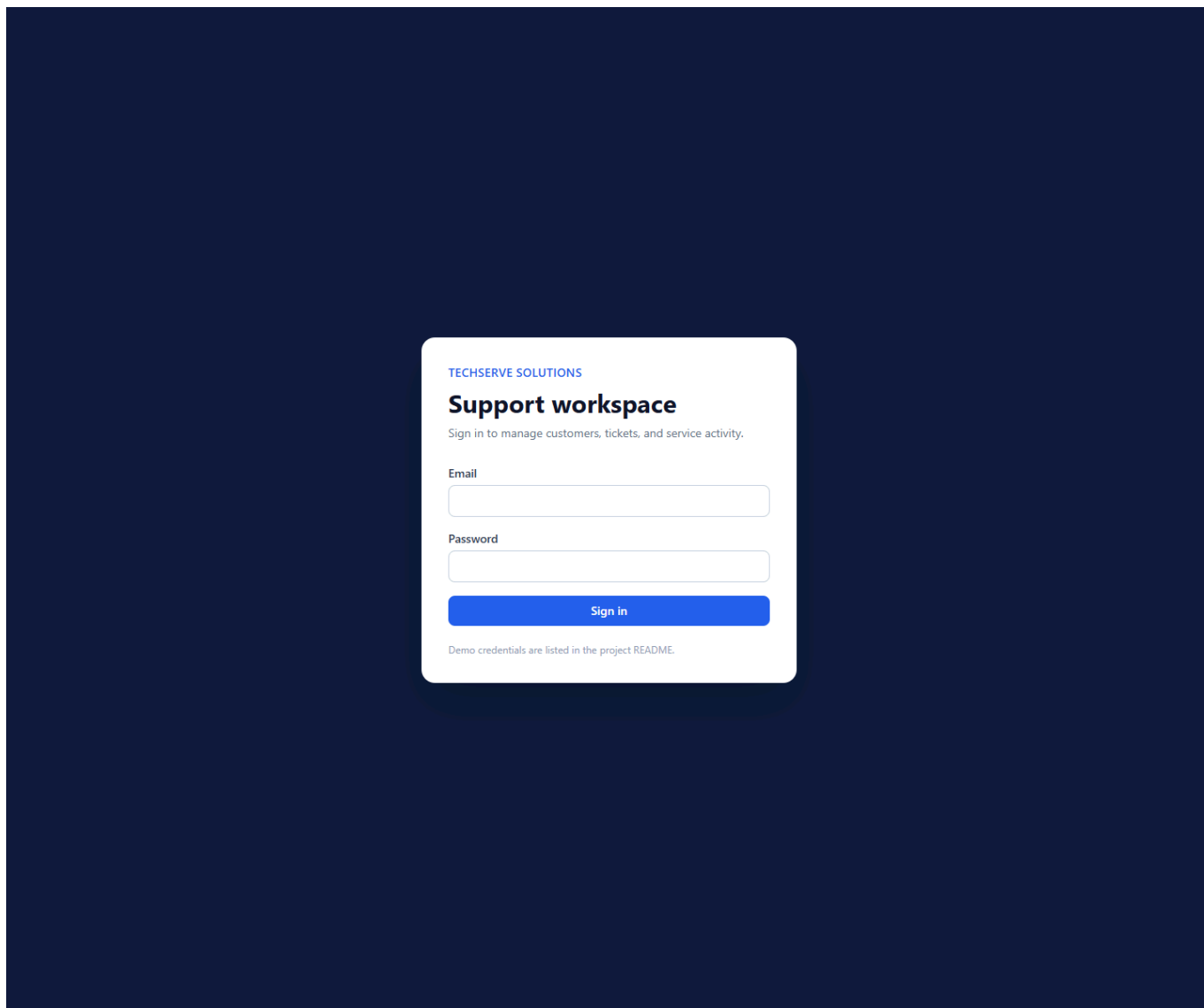


Figure 1. Login page for the TechServe support workspace.

3. System Architecture

The application uses a layered web architecture. The frontend is responsible for user interaction, the backend exposes protected REST endpoints, the database stores persistent CRM records, the AI service generates ticket assistance, and the messaging service attempts SMTP email delivery.

Frontend Layer

The user interface is built with **React**, **Vite**, and **Tailwind CSS**. React Router controls navigation between Login, Dashboard, Customers, Tickets, and Reports pages. An authentication context stores the current user and access token in local storage. An Axios client automatically adds the JWT bearer token to API requests and clears local authentication state when the backend returns an unauthorized response.

Backend Layer

The backend is a **FastAPI** REST API. Route modules separate authentication, customers, tickets, dashboard statistics, reports, and notifications. Pydantic schemas validate incoming payloads and serialize API

responses. Reusable dependencies enforce authentication and permissions on the server, which means security does not depend only on hidden frontend links.

Database Layer

The application uses **SQLite** for persistent storage and **SQLAlchemy ORM** for models, relationships, and transactions. SQLite is appropriate for this assignment MVP because it is easy to initialize locally while preserving data across server restarts. The same ORM-based architecture could later be adapted to a production relational database.

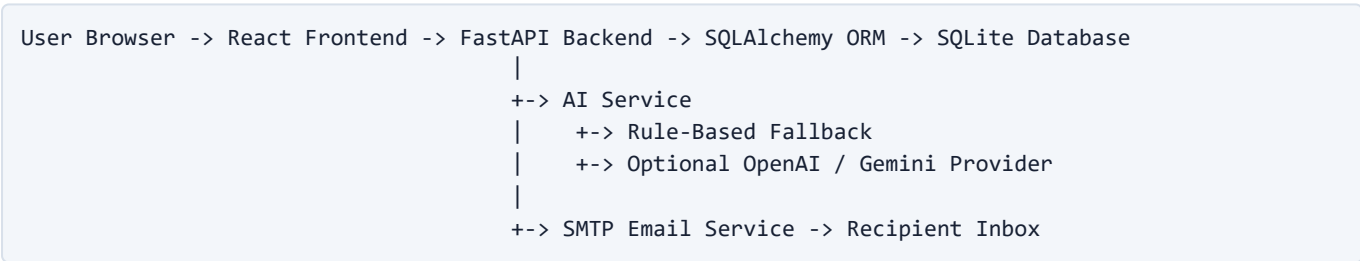
Authentication And Authorization

Login uses **JWT access tokens** signed with a secret loaded from the environment. Passwords are stored as Argon2 hashes, never as plaintext database values. Backend dependencies verify tokens and enforce Manager or Agent access rules. The frontend provides a matching user experience by hiding manager-only navigation for Agent accounts.

AI And Messaging Services

The AI module provides a local rule-based fallback plus optional OpenAI and Gemini adapters. The messaging module uses Python's standard-library SMTP tools to format and send email. Delivery attempts are persisted so ticket work remains traceable even when external email configuration is unavailable or authentication fails.

Text-Based Architecture Diagram



AI-Enhanced CRM System Architecture

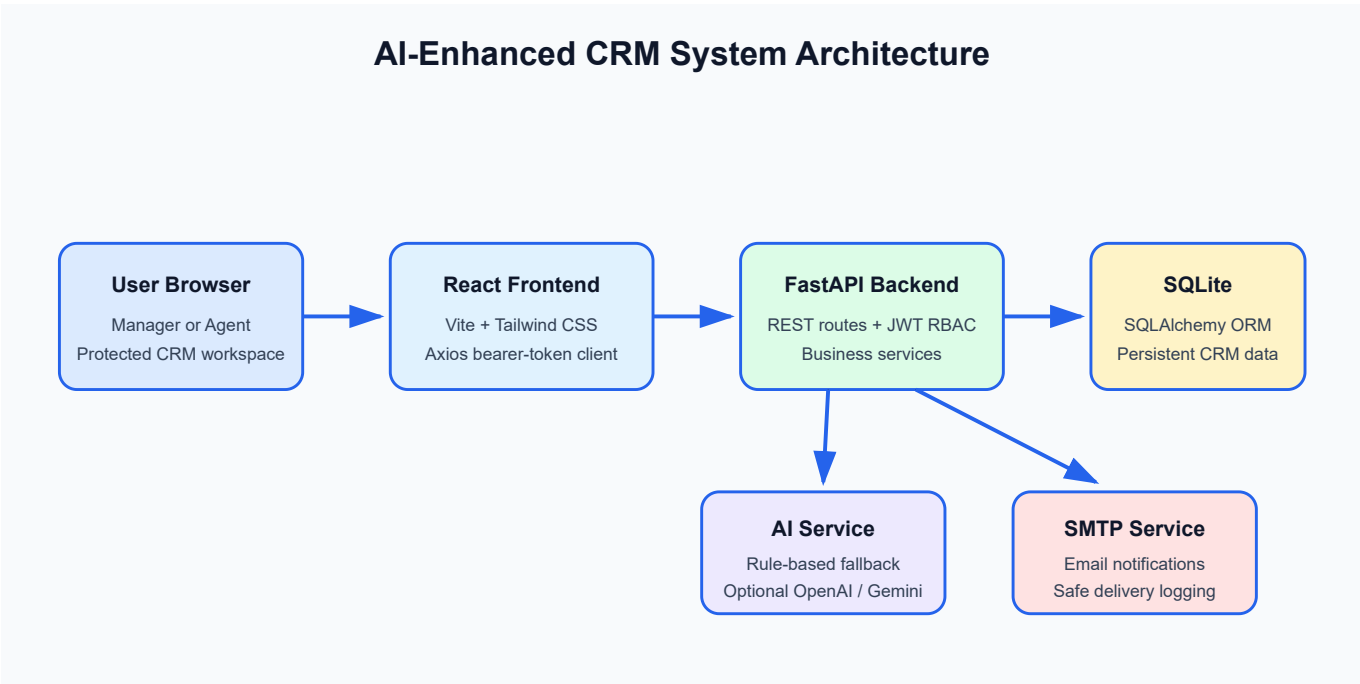


Figure 2. High-level architecture of the AI-enhanced CRM system.

4. Database Schema

The database schema is designed around a ticket-centric CRM workflow. Customer profiles and users provide context and ownership, while comments, activity records, and notifications preserve the history of each support request.

Users / Agents

The users table stores authenticated staff accounts. Important fields include:

- `id`: primary key
- `name`: display name
- `email`: unique login identifier
- `password_hash`: secure hashed password
- `role`: either manager or agent
- `is_active`: controls account availability
- `created_at`: account creation timestamp

One Agent user can be assigned many customers and many tickets. Managers are users with broader permissions rather than a separate database entity.

Customers

The customers table stores CRM profile information:

- `id`: primary key
- `full_name`, `email`, `phone`, `company`: customer contact data
- `notes`: contextual CRM notes
- `assigned_agent_id`: optional foreign key to `users.id`
- `created_at`: profile creation timestamp

One customer can have many support tickets. This allows a customer detail page to show contact information and ticket history together.

Tickets

The tickets table is the central operational entity:

- `id`: primary key
- `title`, `description`: support request content
- `status`: Open, In Progress, Resolved, or Closed
- `priority`: Low, Medium, High, or Critical
- `category`: business-facing category
- `customer_id`: foreign key to the customer
- `assigned_agent_id`: optional foreign key to the responsible Agent
- `created_at`, `updated_at`, `resolved_at`: lifecycle timestamps
- `ai_category`, `ai_sentiment`, `ai_summary`: stored AI outputs

Ticket Comments

The `ticket_comments` table stores chronological support updates and internal notes:

- `id`: primary key
- `ticket_id`: foreign key to the ticket
- `agent_id`: foreign key to the user who added the update
- `message`: update text
- `is_internal`: marks internal support notes
- `created_at`: comment timestamp

Ticket Activity

The `ticket_activities` table is an audit trail:

- `id`: primary key
- `ticket_id`: foreign key to the ticket
- `actor_id`: foreign key to the acting user
- `action`: action description
- `old_value`, `new_value`: changed values when applicable
- `created_at`: action timestamp

It records ticket creation, comments, status changes, priority changes, resolution, AI analysis, and summary refresh actions.

Notification Logs

The `notification_logs` table persists messaging attempts:

- `id`: primary key
- `ticket_id`: foreign key to the ticket
- `platform`: currently smtp
- `message`: stored email subject and body
- `status`: logged, sent, skipped, or failed
- `sent_at`: attempt timestamp

Relationships

- One user/agent can be assigned many customers.
- One user/agent can be assigned many tickets.
- One customer can have many tickets.
- One ticket can have many comments.
- One ticket can have many activity records.
- One ticket can have many notification logs.

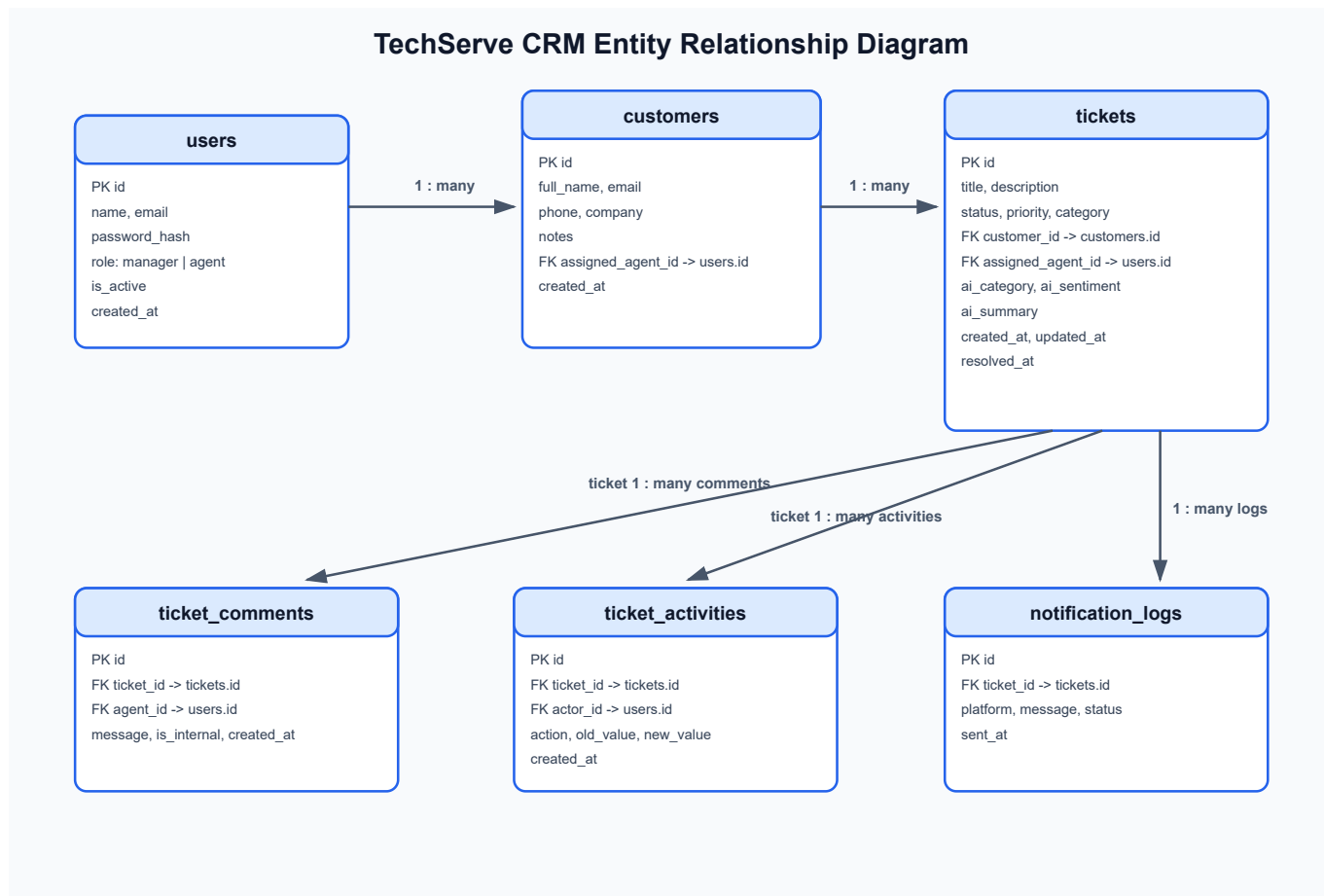


Figure 3. Main entities and relationships in the SQLite database.

5. AI Integration

The AI service improves ticket handling while keeping the application usable without paid API credentials. It supports five practical functions.

Auto Ticket Categorization

When a ticket is created, the system analyzes the title and description and assigns a category such as Account, Technical, Billing, Refund, Shipping, or General. This improves organization and makes dashboard and report statistics more useful.

Sentiment Analysis

The same ticket content is analyzed for sentiment. Supported labels include Positive, Neutral, Negative, and Frustrated. A visibly frustrated request helps the support team identify work that may need quicker attention.

Escalation Recommendation

The application recommends escalation when a frustrated ticket still has Low or Medium priority. This gives the Agent or Manager a prompt to review priority before the issue is overlooked.

AI Reply Suggestion

From the ticket detail page, a user can generate a professional draft response. The suggested reply acknowledges the customer's issue, states that the support team is reviewing it, and avoids inventing a resolution. The draft can be copied into the comment box for editing.

AI Resolution Summary

When a ticket becomes Resolved, the system creates a concise internal summary. A user can also refresh the summary manually. The summary captures issue context, category, current status, and logged updates.

Trigger Points And Provider Strategy

AI runs during ticket creation, manual ticket analysis, reply generation, ticket resolution, and manual summary refresh. Outputs are saved where appropriate and shown directly in the ticket-detail interface.

The default `rule_based` provider works locally without API keys. Optional OpenAI and Gemini providers can be enabled through environment variables. If a hosted provider is unavailable, lacks credentials, or returns an error, the local fallback produces a useful result instead of breaking the ticket workflow.

Example Input And Output

Example ticket

Title: Cannot login to my account
Description: I am very frustrated. I reset my password but nothing works.

Example AI output

Category: Account or Technical
Sentiment: Frustrated
Suggested reply: A professional support response acknowledging the login issue and confirming review.
Summary: A short internal record of the account-access issue, status, and logged support updates.

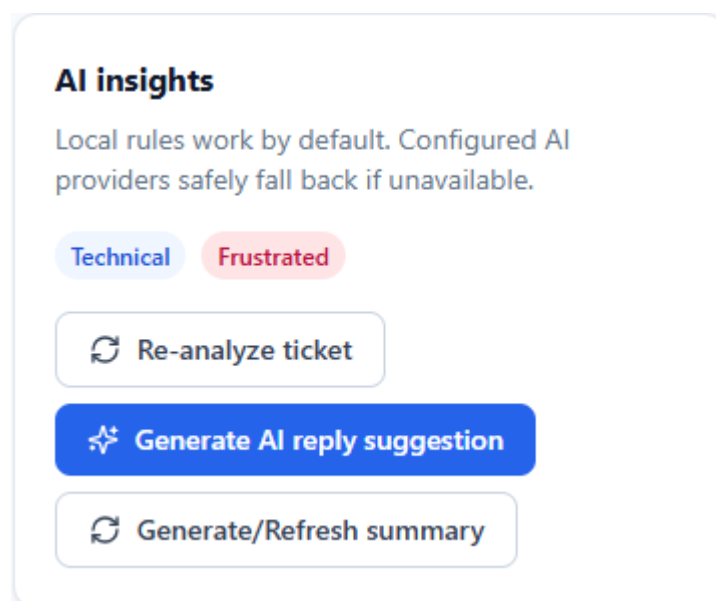


Figure 4. AI category and sentiment shown inside the ticket workflow.

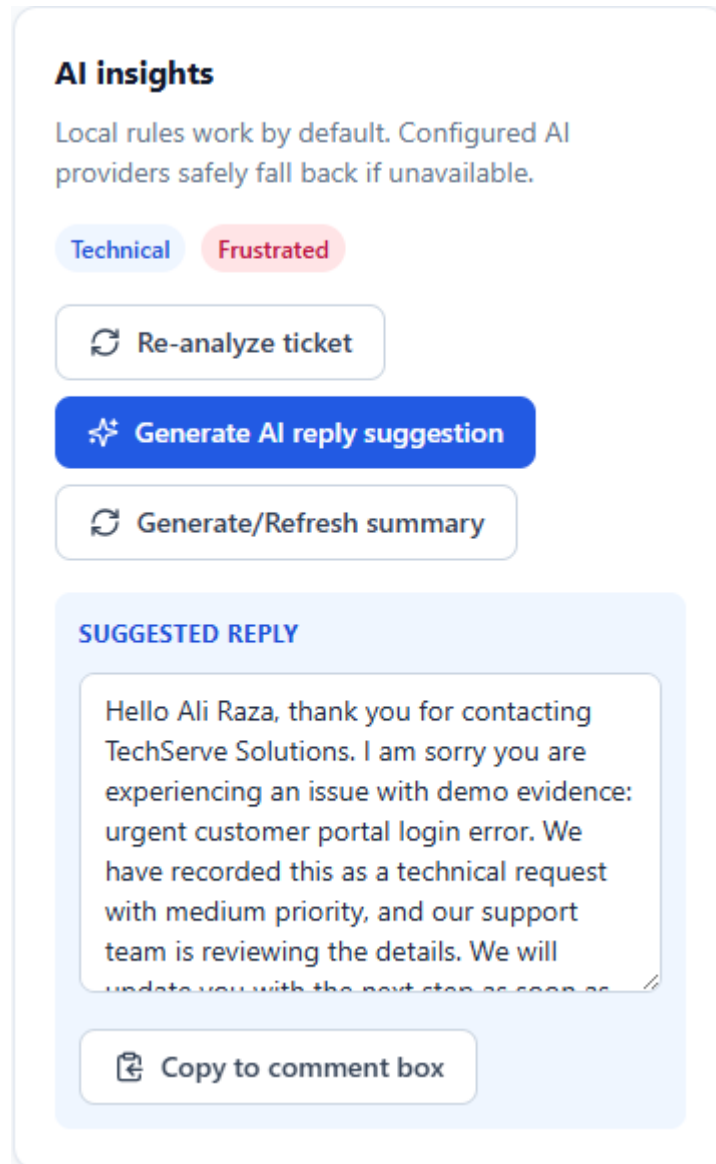


Figure 5. Generated AI reply suggestion available to the support user.

6. Messaging Integration

SMTP email was selected as the messaging platform because it provides a practical, widely supported way to send real notifications without tying the assignment to one proprietary messaging application. The implementation uses Python's `smtplib` module and `EmailMessage`.

Email delivery is triggered when:

- a new ticket is created;
- a ticket is escalated to Critical priority;
- a ticket is resolved.

Each message includes the ticket ID, title, customer, priority, status, category, assigned agent, and event type. Every attempt is saved in `notification_logs`, allowing Managers and assigned Agents to review delivery history from the ticket detail page.

Safe Failure Handling

Messaging is deliberately fault tolerant. If messaging is disabled, the event is recorded with a logged status. If configuration is incomplete, the event is skipped. If SMTP delivery raises an error, the event is stored as failed. A delivery problem never prevents ticket creation, priority updates, or resolution.

During the initial evidence run, the SMTP connection was attempted for ticket creation, Critical escalation, and resolution. The configured SMTP server rejected authentication, and the application correctly stored failed entries without exposing credentials or interrupting ticket work. After the provider App Password was refreshed, the backend was restarted and a Manager test notification was delivered successfully. The received message is shown in Figure 20.

SMTP Setup Summary

```
ENABLE_MESSAGING=true  
MESSAGING_PLATFORM=smtp  
SMTP_HOST=[SMTP server hostname]  
SMTP_PORT=587  
SMTP_USERNAME=[SMTP account username]  
SMTP_PASSWORD=[SMTP password or provider app password]  
SMTP_FROM_EMAIL=[sender email]  
SMTP_TO_EMAIL=[recipient email]  
SMTP_USE_TLS=true
```

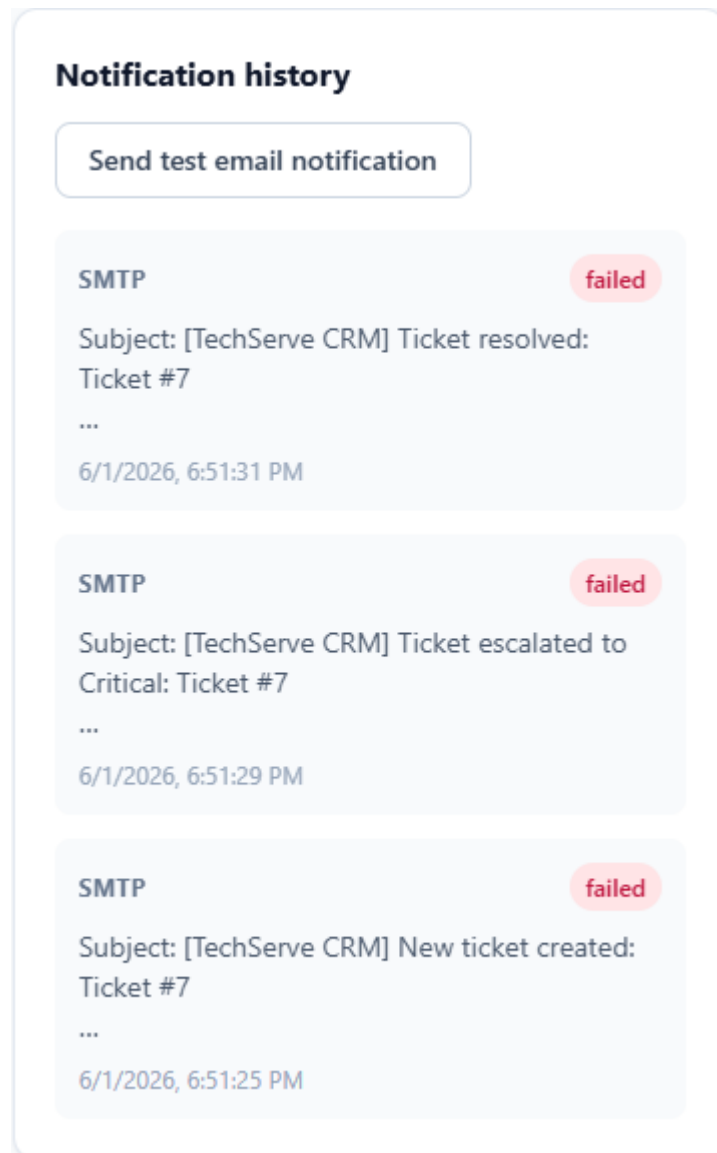


Figure 6. Notification history showing SMTP attempt outcomes for ticket events.

Received email evidence: Figure 20 shows the successfully delivered SMTP test notification.

7. Feature Screenshots

The following screenshots were captured from the local application using seeded demo data and a temporary evidence ticket. The temporary ticket was removed after capture.

Login Page

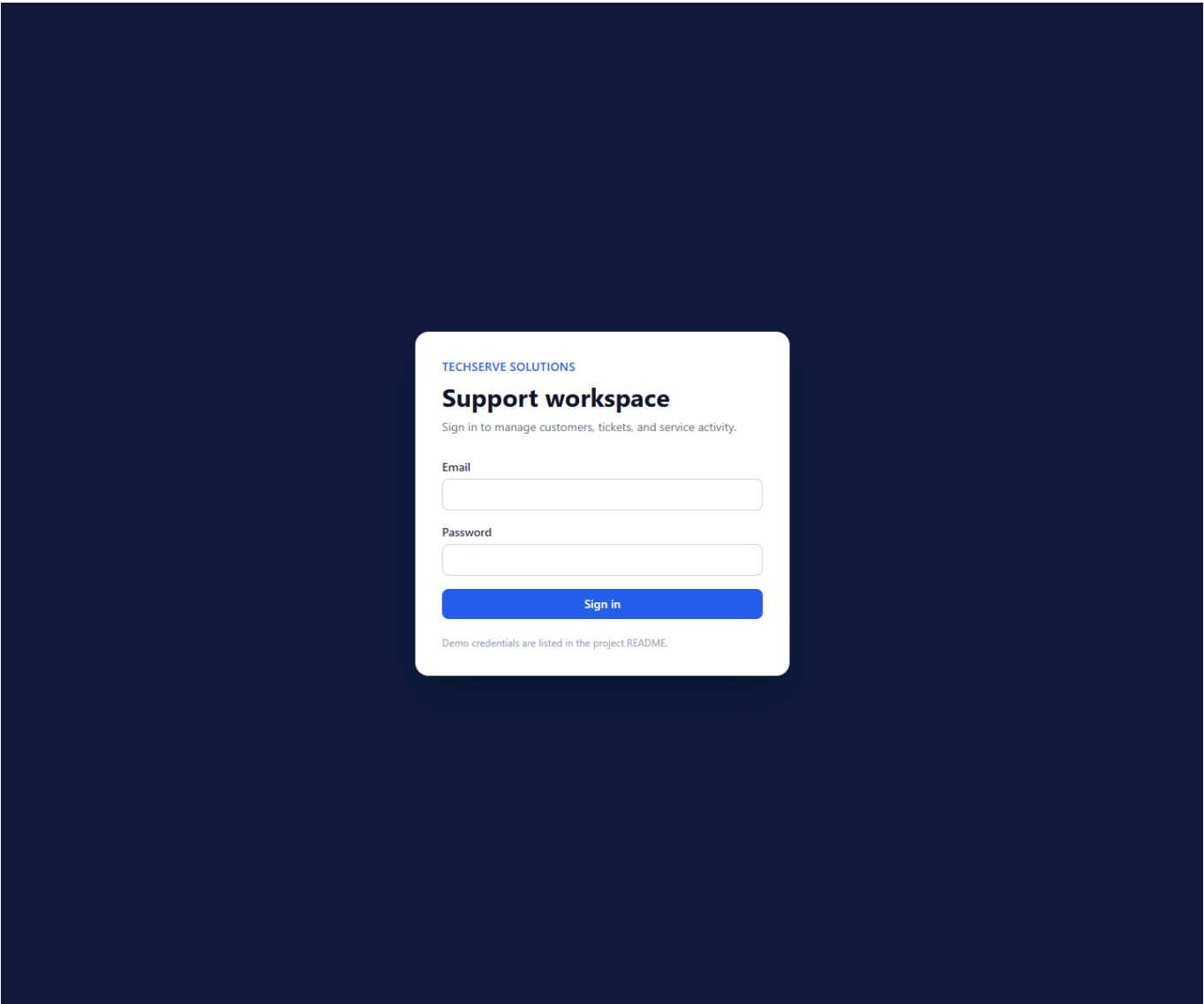


Figure 7. Secure login screen for Manager and Agent accounts.

Dashboard

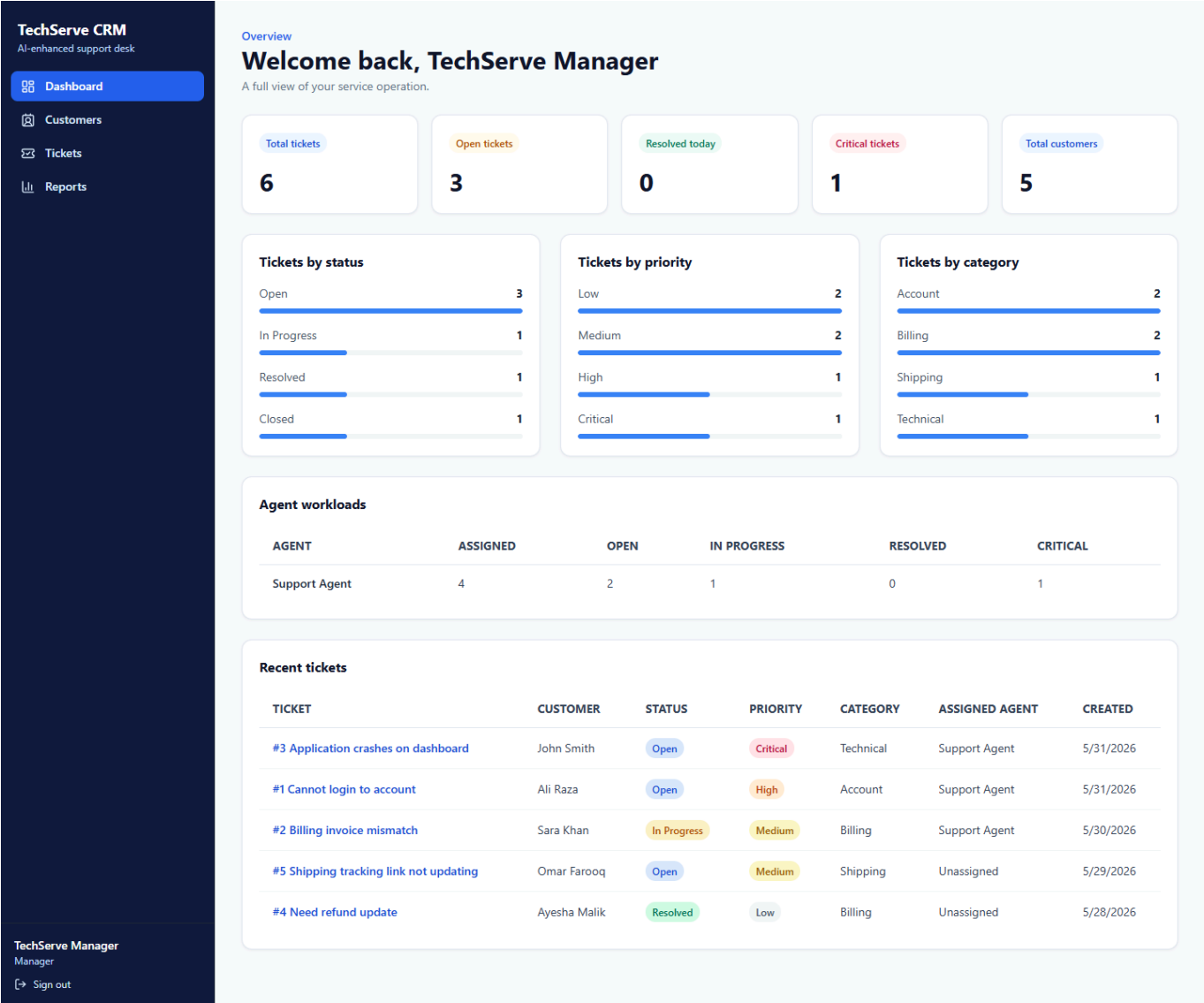


Figure 8. Manager dashboard with ticket counts, visual indicators, workload metrics, and recent tickets.

Customers List

TechServe CRM
AI-enhanced support desk

Dashboard

Customers

Tickets

Reports

TechServe Manager

Manager

Sign out

Customers

Search profiles and review their service history.

+ Add customer

Search name, email, or company

FULL NAME	EMAIL	PHONE	COMPANY	DATE ADDED	ACTIONS
Omar Farooq	omar.farooq@example.com	+92 321 1112233	FinEdge	6/1/2026	
Ayesha Malik	ayesha.malik@example.com	+92 333 4567890	RetailPro	6/1/2026	
John Smith	john.smith@example.com	+1 415 555 0147	CloudNova	6/1/2026	
Sara Khan	sara.khan@example.com	+92 301 7654321	BrightSoft	6/1/2026	
Ali Raza	ali.raza@example.com	+92 300 1234567	TechServe Solutions	6/1/2026	

Figure 9. Searchable customer list with contact details and actions.

Customer Detail

TechServe CRM

AI-enhanced support desk

Dashboard

Customers

Tickets

Reports

TechServe Manager

Manager

Sign out

TechServe Solutions

Ali Raza

ali.raza@example.com | +92 300 1234567

Edit profile

Customer details

EMAIL
ali.raza@example.com

PHONE
+92 300 1234567

COMPANY
TechServe Solutions

ASSIGNED AGENT
Support Agent

DATE ADDED
6/1/2026

Notes
Primary contact for internal technology support.

Ticket history

+ New ticket

#1 Cannot login to account

5/31/2026, 1:31:07 PM

Open High

#6 Profile company name correction

5/26/2026, 1:31:07 PM

Closed Low

Figure 10. Customer profile with CRM information and ticket history.

Tickets List

TechServe CRM

AI-enhanced support desk

Dashboard

Customers

Tickets

Reports

TechServe Manager

Manager

Sign out

Tickets

Track requests from intake through resolution.

+ New ticket

Search title, description, category, or customer

All statuses

All priorities

All agents

TITLE	CUSTOMER	STATUS	PRIORITY	CATEGORY	ASSIGNED AGENT	DATE CREATED	ACTIONS
#3 Application crashes on dashboard	John Smith	Open	Critical	Technical	Support Agent	5/31/2026	
#1 Cannot login to account	Ali Raza	Open	High	Account	Support Agent	5/31/2026	
#2 Billing invoice mismatch	Sara Khan	In Progress	Medium	Billing	Support Agent	5/30/2026	
#5 Shipping tracking link not updating	Omar Farooq	Open	Medium	Shipping	Unassigned	5/29/2026	
#4 Need refund update	Ayesha Malik	Resolved	Low	Billing	Unassigned	5/28/2026	
#6 Profile company name correction	Ali Raza	Closed	Low	Account	Support Agent	5/26/2026	

Figure 11. Ticket list with filters, statuses, priorities, categories, and assigned agents.

Ticket Detail

TechServe CRM

AI-enhanced support desk

Dashboard

Customers

Tickets

Reports

TechServe Manager

Manager

Sign out

Ticket #7

Demo evidence: urgent customer portal login error

Open

Medium

Frustrated

Comment added successfully.

Request

The customer is frustrated because the login portal crashes after a password reset. This is urgent and nothing works.

Comments

1 updates

TechServe Manager

6/1/2026, 6:51:25 PM

Demo evidence update: issue reproduced and queued for resolution.

Internal note

Add update

Add an update or internal note

☒ Internal note

Add comment

Ticket details

Customer

Ali Raza

Customer email

ali.raza@example.com

Agent

Support Agent

Category

Technical

Created

6/1/2026, 6:51:22 PM

Update status

Status

Open

Priority

Medium

Save changes

AI insights

Local rules work by default. Configured AI providers safely fall back if unavailable.

Technical

Frustrated

Re-analyze ticket

Generate AI reply suggestion

Generate/Refresh summary

Notification history

Send test email notification

SMTP

failed

Subject: [TechServe CRM] New ticket created: Ticket #7

...

6/1/2026, 6:51:25 PM

Activity log

Comment added

TechServe Manager | 6/1/2026, 6:51:25 PM

Ticket created

TechServe Manager | 6/1/2026, 6:51:22 PM

Figure 12. Ticket detail page with request content, ownership, comments, AI tools, and history.

AI Insights

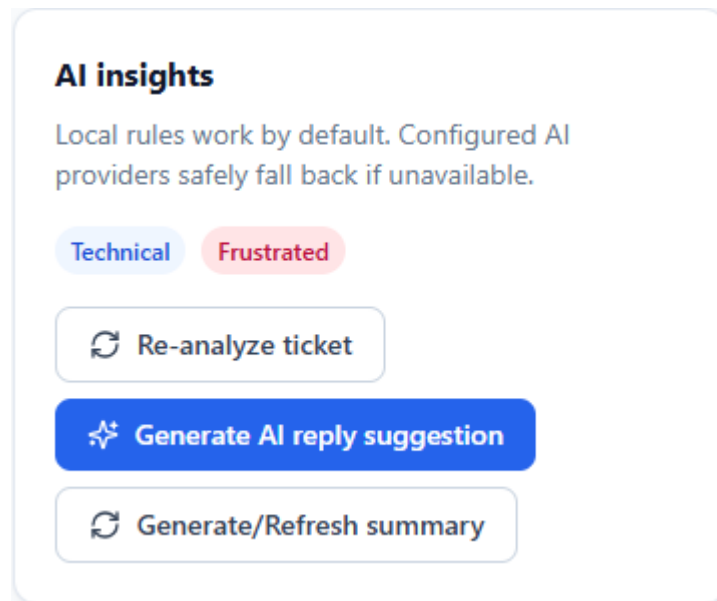


Figure 13. AI categorization and sentiment analysis.

AI Reply Suggestion

AI insights

Local rules work by default. Configured AI providers safely fall back if unavailable.

Technical Frustrated

 Re-analyze ticket

 **Generate AI reply suggestion**

 Generate/Refresh summary

SUGGESTED REPLY

Hello Ali Raza, thank you for contacting TechServe Solutions. I am sorry you are experiencing an issue with demo evidence: urgent customer portal login error. We have recorded this as a technical request with medium priority, and our support team is reviewing the details. We will update you with the next step as soon as

 Copy to comment box

Figure 14. Generated customer-service reply draft.

Activity Log

Activity log

Ticket resolved
TechServe Manager | 6/1/2026, 6:51:29 PM
Open to Resolved

Status changed
TechServe Manager | 6/1/2026, 6:51:29 PM
Open to Resolved

Priority changed
TechServe Manager | 6/1/2026, 6:51:26 PM
Medium to Critical

Comment added
TechServe Manager | 6/1/2026, 6:51:25 PM

Ticket created
TechServe Manager | 6/1/2026, 6:51:22 PM

Figure 15. Ticket activity trail with timestamps and changed values.

Notification History

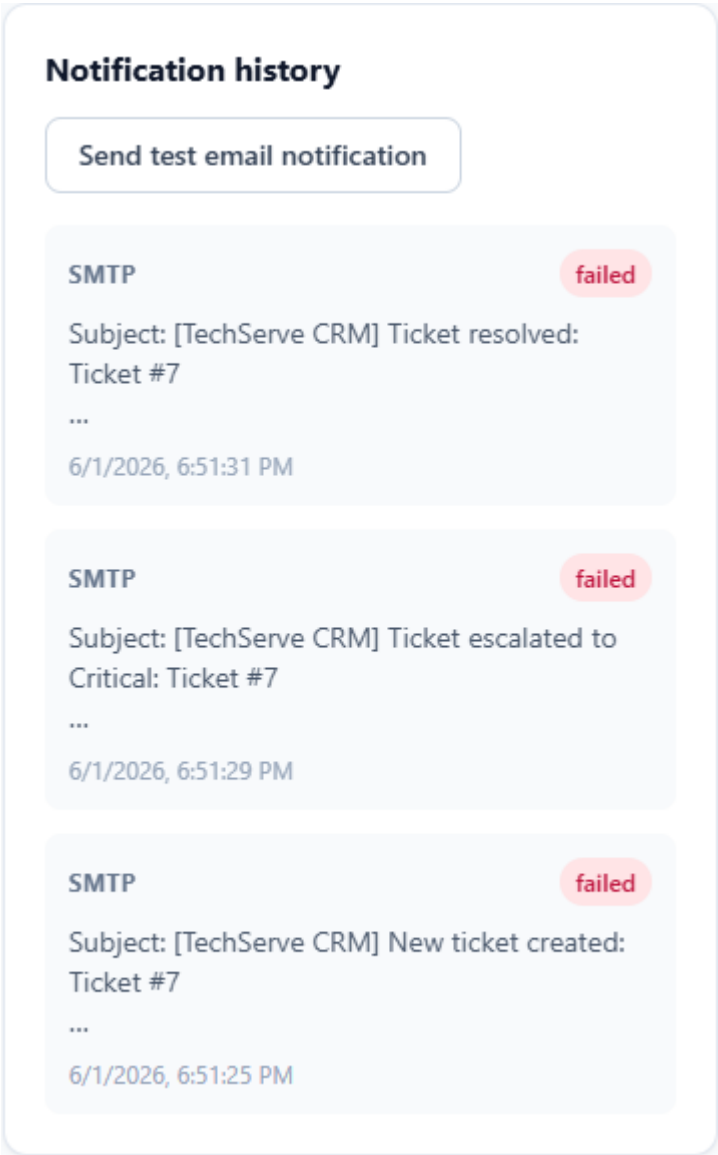


Figure 16. SMTP notification attempt history.

Reports Page

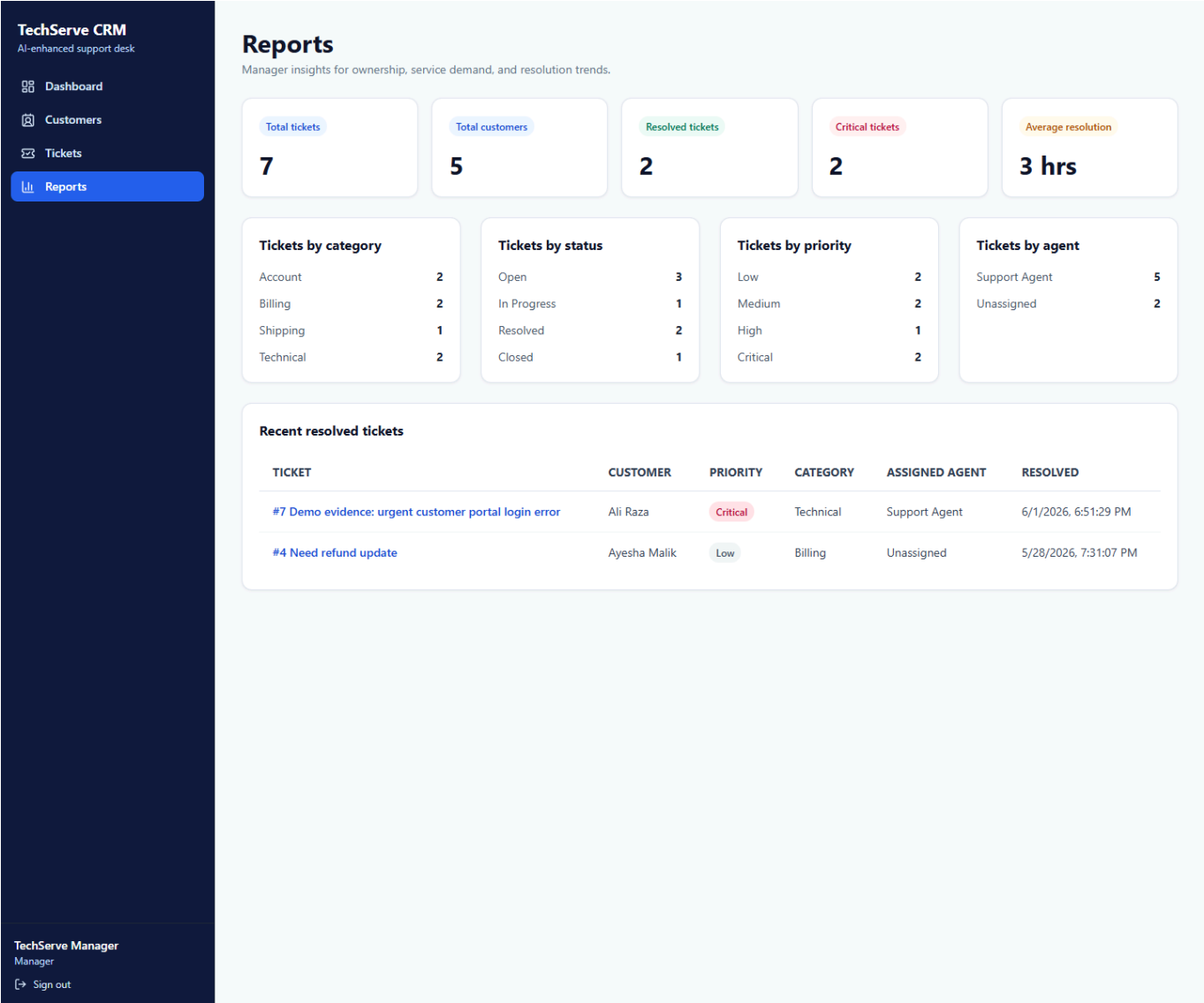


Figure 17. Manager-only report summary.

Agent Dashboard And Restricted Reports Evidence

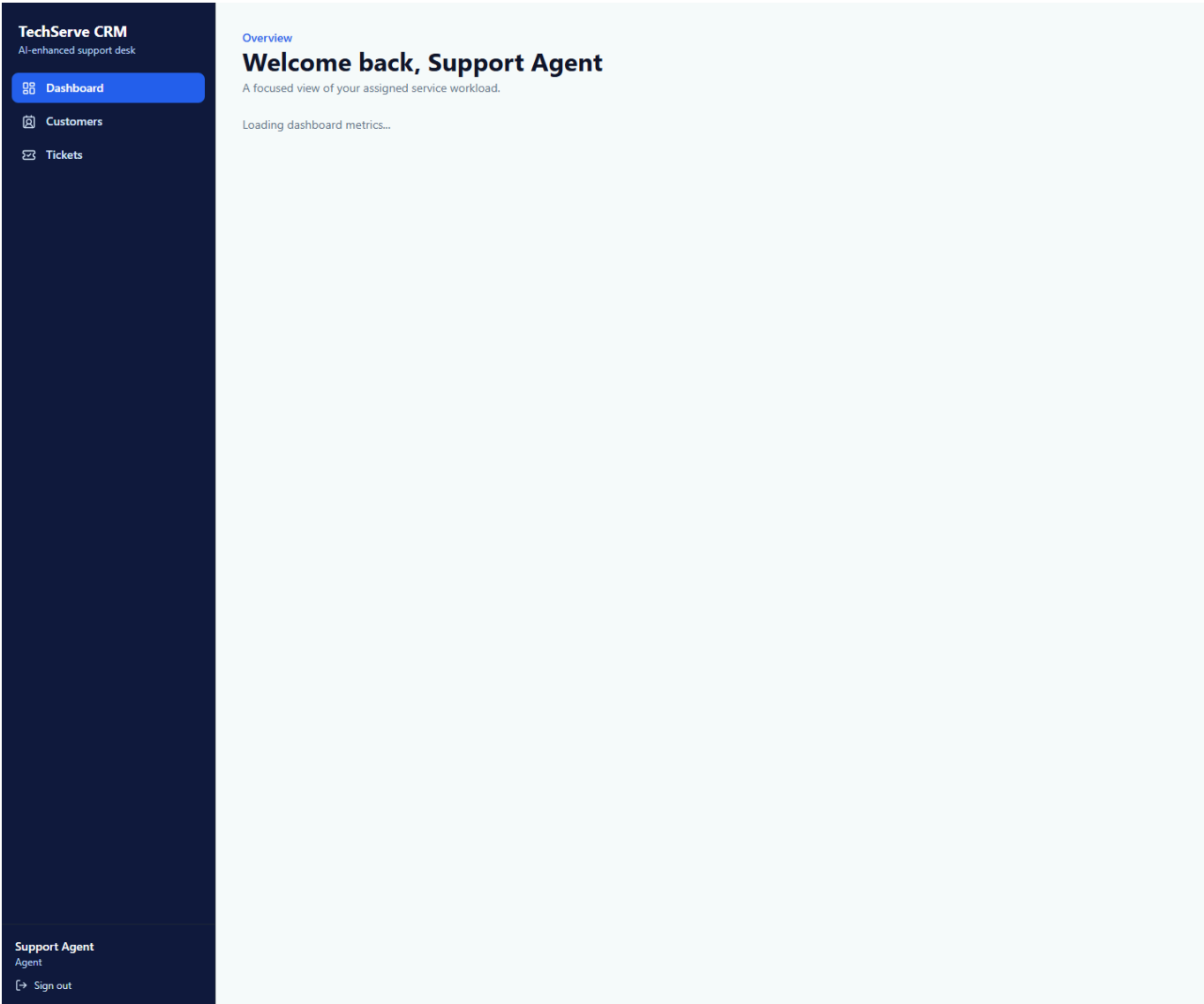


Figure 18. Scoped Agent dashboard.

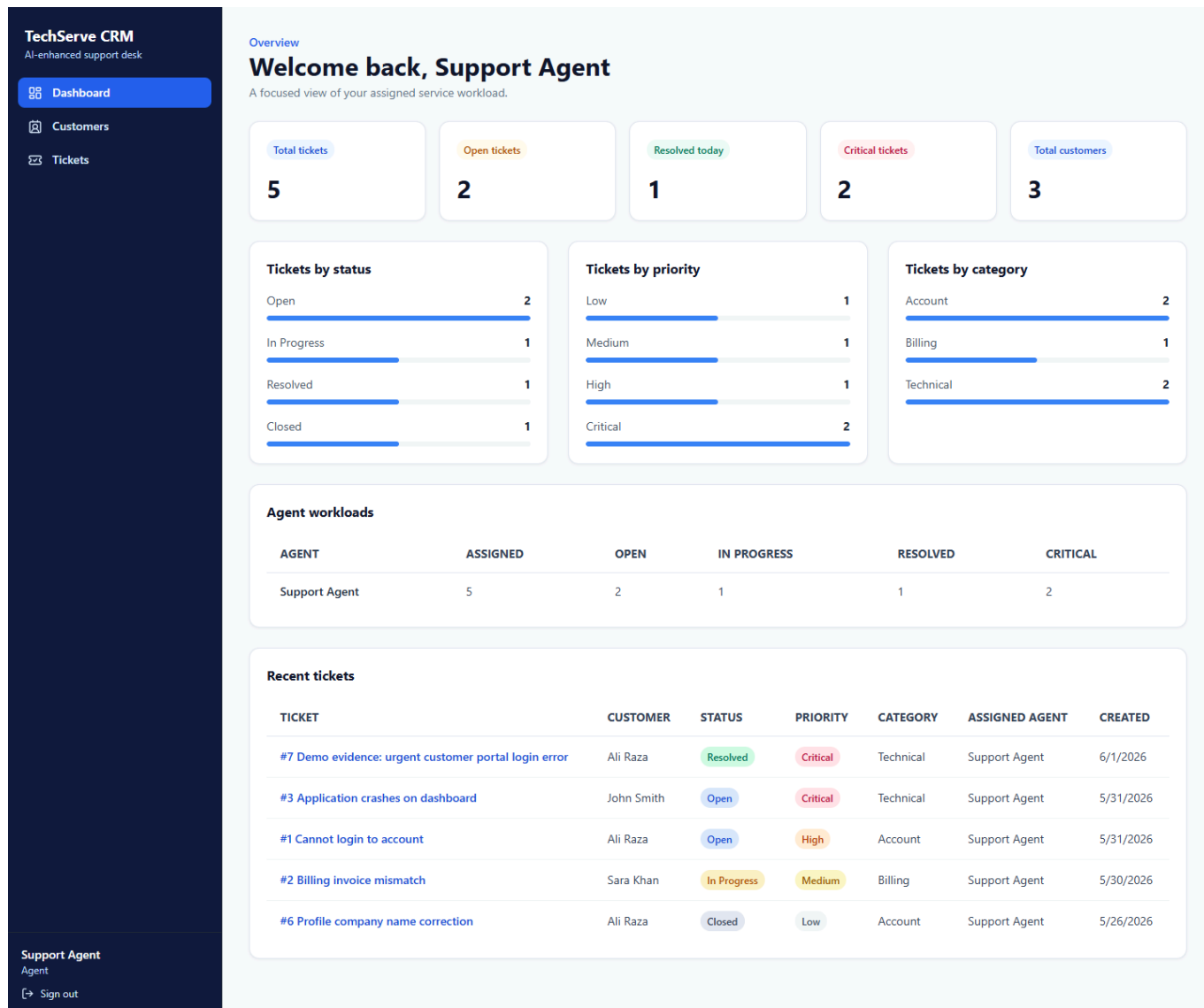


Figure 19. Agent view after attempting to open the manager-only Reports route. The Reports link is not shown and the application redirects to the Agent dashboard.

Email Notification

[TechServe CRM] Test email notification: Ticket #1 Inbox x



msarimkhanghouri@gmail.com

to me ▼

Test email notification

Ticket ID: 1

Title: Cannot login to account

Customer: Ali Raza

Priority: High

Status: Open

Category: Account

Assigned agent: Support Agent

Event type: ticket_test

Figure 20. Successfully delivered SMTP test notification received in the configured inbox.

8. Challenges & Learnings

Designing Role-Based Access

The application needed different experiences for Managers and Agents. Hiding a Reports link is not enough for security, so backend dependencies also enforce permissions. Agents receive restricted data from the API, cannot delete customers or tickets, cannot reassign tickets, and cannot access manager reports.

Keeping Ticket Workflows Safe

AI and messaging are useful additions, but external services can fail. The design keeps the core CRM workflow independent from provider availability. Rule-based AI continues working without paid API keys, and SMTP errors become persistent notification records instead of uncaught failures.

Maintaining Database Relationships

Customers, tickets, users, comments, activity records, and notifications have connected lifecycles. SQLAlchemy relationships keep the model readable and ensure that deleting a ticket also removes its dependent comments, activities, and notification logs.

Making AI Practical

The project demonstrates that AI features should support concrete user actions. Categorization, sentiment, reply suggestions, and summaries are visible where Agents already work. The fallback strategy also makes the project demonstrable in a local environment.

Testing Separate User Roles

Manager and Agent workflows were tested independently. This exposed the importance of verifying both UI behavior and direct API restrictions. A professional support system must remain secure even when someone bypasses the frontend and calls backend routes directly.

Protecting Configuration

Secrets belong in ignored `.env` files, not source code or screenshots. The repository uses example environment files with placeholders. SMTP evidence is captured through application status history without exposing usernames, passwords, tokens, or unrelated inbox content.

9. Work Division

The project was completed by Group 4, BCS-6J. Contributions were divided as follows:

Name	Student ID	Contribution
Muhammad Sarim Khan Ghouri	23k-0720	Project coordination, backend integration review, AI workflow testing, SMTP/email evidence, final demo preparation, and report compilation.
Syeda Hamna Uraizee	22k-5112	Frontend interface review, customer and ticket workflow testing, dashboard and reports validation, screenshot collection, and documentation support.
Sohaib Ahmer	22k-4784	Backend module testing, authentication and role-based access verification, database schema review, notification testing, and README/GitHub verification.

10. References

1. FastAPI. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
2. React. *React Documentation*. <https://react.dev/>
3. Vite. *Vite Guide*. <https://vite.dev/guide/>
4. Tailwind CSS. *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
5. SQLAlchemy. *SQLAlchemy Documentation*. <https://docs.sqlalchemy.org/>
6. PyJWT. *PyJWT Documentation*. <https://pyjwt.readthedocs.io/>
7. Python Software Foundation. *smtplib - SMTP Protocol Client*. <https://docs.python.org/3/library/smtplib.html>
8. Python Software Foundation. *email.message.EmailMessage*. <https://docs.python.org/3/library/email.message.html>
9. OpenAI. *Responses API Reference*. <https://platform.openai.com/docs/api-reference/responses>
10. Google. *Gemini API GenerateContent Documentation*. <https://ai.google.dev/api/generate-content>
11. Pydantic. *Pydantic Documentation*. <https://docs.pydantic.dev/>

12. Axios. *Axios Documentation*. <https://axios-http.com/docs/intro>

13. React Router. *React Router Documentation*. <https://reactrouter.com/>